

Table of Contents

Introduction.....	2
Three ways of Conditional rendering.....	3
Splitting the component from the page.....	4
<i>Conditional rendering of ItemGrid or ItemTable.....</i>	<i>7</i>
<i>Fixing the lagging backend.....</i>	<i>11</i>
Toggling descriptions and images in ItemTable.....	12
Filtering content.....	14
Carousel in HomePage.....	16

Demo: Conditional Rendering

Author: Baifan Zhou

Detailed explanations are created with the help of ChatGPT.

Introduction

Conditional rendering in React enables dynamic control over what gets displayed to the user based on state, props, or other criteria. This demo explores three primary methods of achieving conditional rendering:

- **If Statement:** Traditional if statements determine which set of JSX elements to render based on conditions.
- **Ternary Operator:** This concise approach uses the format `condition ? true : false` to render elements based on a condition. For example, button text might change depending on a loading state.
- **Logical AND (&&):** The logical AND operator renders content only if a condition is true, such as displaying an error message when an error value is present.

Splitting the Component

To enhance clarity, the demo splits `ItemListPage` into the `ItemTable` component, responsible only for the `ItemLable`, and the `ItemListPage` component, responsible for the page.

ItemGrid Component

A new component, `ItemGrid`, is added alongside `ItemTable`. `ItemListPage.jsx` now conditionally renders either `ItemTable` or `ItemGrid` based on a state variable. This approach involves:

Toggling Columns in ItemTable

Additional functionality is introduced to `ItemTable.jsx` to toggle the visibility of item descriptions and images.

Filtering Content

A search bar is integrated to filter items based on the search query. Changes to `ItemListPage.jsx` include:

Carousel in HomePage

Finally, a carousel component is added to `HomePage.jsx` using `react-bootstrap`.

Three ways of Conditional rendering

Conditional rendering in React refers to the practice of rendering different elements or components based on certain conditions. It allows you to dynamically control what gets displayed to the user based on the state, props, or other criteria. We have seen this a bit in our previous demo of AJAX Http Get, and will see more in this demo.

There are three ways to achieve it:

if Statement: You can use traditional if statements to conditionally render elements. For example, in a function, you might return one set of JSX if a condition is true, and another set if it is false. Example:

```
function MyComponent({ isLoggedIn }) {  
  if (isLoggedIn) {  
    return <h1>Welcome back!</h1>;  
  } else {  
    return <h1>Please sign up.</h1>;  
  }  
}
```

Ternary Operator: A more concise way is using the ternary operator (condition ? true : false).

In ItemListPage.jsx: changing the button text depending on the Boolean value of loading

```
37 <Button onClick={fetchItems} className="btn btn-primary mb-3" disabled={loading}>  
38   {loading ? 'Loading...' : 'Refresh Items'}  
39 </Button>
```

Logical AND (&&): This operator is used to render something only if a condition is true. It is useful for simple conditions.

In ItemListPage.jsx: display the error message if the error value is not false.

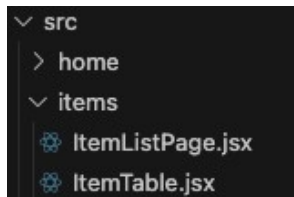
```
40 {error && <p style={{ color: 'red' }}>{error}</p>}
```

Splitting the component from the page

Now we are going to add a new component, ItemGrid. The ItemListPage.jsx should display the items in either Table or Grid, depending on a state.

Before adding ItemGrid, we first need to split the current code of ItemListPage.jsx into: (1) a page component that is responsible for the rendering and the logic for the ItemListPage, and (2) a Table component that is only responsible for the ItemTable.

We create a file ItemTable.jsx under myshop/src/items



Change the ItemTable.jsx to:

```
ItemTable.jsx
myshop > src > items > ItemTable.jsx > ...
1  import React from 'react';
2  import { Table } from 'react-bootstrap';
3
4  const ItemTable = ({ items, apiUrl }) => {
5    return (
6      <Table striped bordered hover>
7        <thead>
8          <tr>
9            <th>ID</th>
10           <th>Name</th>
11           <th>Price</th>
12           <th>Description</th>
13           <th>Images</th>
14         </tr>
15       </thead>
16       <tbody>
17         {items.map(item => (
18           <tr key={item.itemId}>
19             <td>{item.itemId}</td>
20             <td>{item.name}</td>
21             <td>{item.price} NOK</td>
22             <td>{item.description}</td>
23             <td><img src={` ${apiUrl}${item.imageUrl}`} alt={item.name} width="120" /></td>
24           </tr>
25         ))}
26       </tbody>
27     </Table>
28   );
29 };
30
31 export default ItemTable;
```

We can see that the ItemTable now becomes a simple component, which takes two input arguments items and apiUrl.

Change the ItemListPage.jsx to:

```
ItemListPage.jsx x ItemTable.jsx
myshop > src > items > ItemListPage.jsx > ...
1 import React, { useState, useEffect } from 'react';
2 import { Button } from 'react-bootstrap';
3 import ItemTable from './ItemTable';
4
5 const API_URL = 'http://localhost:5043'
6
7 const ItemListPage = () => {
8   const [items, setItems] = useState([]);
9   const [loading, setLoading] = useState(false);
10  const [error, setError] = useState(null);
11
12  const fetchItems = async () => {
13    setLoading(true); // Set loading to true when starting the fetch
14    setError(null); // Clear any previous errors
15
16    try {
17      const response = await fetch(`${API_URL}/api/itemapi/itemlist`);
18      if (!response.ok) {
19        throw new Error('Network response was not ok');
20      }
21      const data = await response.json();
22      setItems(data);
23      console.log(data);
24    } catch (error) {
25      console.error('There was a problem with the fetch operation: ${error.message}');
26      setError('Failed to fetch items.');
```

Line3 imports the ItemTable component from the ./ItemTable.jsx file. The jsx extension is omitted because modern JavaScript tooling (e.g., Webpack, Babel) and module bundlers are configured to automatically resolve file extensions.

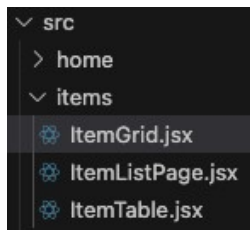
Line42 uses the ItemTable component, passing two arguments into it.

Now we see that the long file ItemListPage.jsx becomes much clear. This separation helps maintain a clean structure and promotes better reusability and readability.

The effect of this code is exact the same as previous code and there will be no visible difference.

Conditional rendering of ItemGrid or ItemTable

Now we add a new file ItemGrid.jsx under myshop/src/items



Change it to:

```
ItemGrid.jsx x
myshop > src > items > ItemGrid.jsx > ItemGrid
1  import React from 'react';
2  import { Card, Col, Row } from 'react-bootstrap';
3
4  const ItemGrid = ({ items, apiUrl }) => {
5
6    return (
7      <div>
8        <Row xs={1} sm={2} md={3} lg={4} className="g-4">
9          {items.map(item => (
10            <Col key={item.itemId}>
11              <Card className="h-100">
12                <Card.Img variant="top" src={` ${apiUrl}${item.imageUrl}`} alt={item.name} />
13                <Card.Body>
14                  <Card.Title>{item.name}</Card.Title>
15                  <Card.Text>
16                    {item.description}
17                  </Card.Text>
18                  <Card.Text>
19                    Price: {item.price} NOK
20                  </Card.Text>
21                </Card.Body>
22              </Card>
23            </Col>
24          ))}
25        </Row>
26      </div>
27    );
28  };
29
30  export default ItemGrid;
```

Imports: Card, Col, Row are components from the react-bootstrap library used for layout and styling.

const ItemGrid defines a functional component that takes items and apiUrl as input arguments (common referred to as properties, or props, in React).

<Row>: A Bootstrap component that creates a row in the grid. The responsive props (xs, sm, md, lg) define how many columns the row should have at different screen sizes.

<Col>: A Bootstrap component that represents a column within a row. Each item is placed in its own column.

<Card>: A Bootstrap component used to display content in a card format. Each item is represented as a card with an image, title, description, and price.

export default ItemGrid: Exports the ItemGrid component as the default export from this module, allowing it to be imported and used in other parts of the application.

<Card.Body>: Contains the main content of the card, including:

- **<Card.Title>**: Displays the item's name.
- **<Card.Text>**: Displays additional text, including description and price.

Also change the ItemListPage.jsx to:

```

ItemLoginPage.jsx x
myshop > src > items > ItemLoginPage.jsx > ...
1  import React, { useState, useEffect } from 'react';
2  import { Button } from 'react-bootstrap';
3  import ItemTable from './ItemTable';
4  import ItemGrid from './ItemGrid';
5
6  const API_URL = 'http://localhost:5043'
7
8  const ItemListPage= () => {
9    const [items, setItems] = useState([]);
10   const [loading, setLoading] = useState(false);
11   const [error, setError] = useState(null);
12   const [showTable, setShowTable] = useState(true);
13
14   const toggleTableOrGrid = () => setShowTable(prevShowTable => !prevShowTable);
15
16   const fetchItems = async () => { ...
34   };
35
36   useEffect(() => { ...
38   }, []);
39
40   return (
41     <div>
42       <h1>Items</h1>
43       <Button onClick={fetchItems} className="btn btn-primary mb-3 me-2" disabled={loading}>
44         {loading ? 'Loading...' : 'Refresh Items'}
45       </Button>
46       <Button onClick={toggleTableOrGrid} className="btn btn-primary mb-3 me-2">
47         {showTable ? 'Display Grid' : 'Display Table'}
48       </Button>
49       {error && <p style={{ color: 'red' }}>{error}</p>}
50       {showTable
51         ? <ItemTable items={items} apiUrl={API_URL} />
52         : <ItemGrid items={items} apiUrl={API_URL} />}
53     </div>
54   );
55 };
56
57 export default ItemListPage;

```

Line4 imports the **ItemGrid** component.

Line12 adds a state “**showTable**” for showing the **ItemTable** (when true) or the **ItemGrid** (when false).

Line14 adds a function that toggles the true/false value of **showTable**. We use the keyword “const” to define the function instead of the keyword “function” for several reasons: (1) it is more concise and aligns with modern JS principles; (2) the **toggleTableOrGrid** becomes an immutable function which is more stable; (3) const is block-scoped, only available to the component **ItemListPage**. functions are hoisted, meaning they move to the top of their scope during compilation.

When updating React state based on its current value, use the function form:

`setShowTable(prevShowTable => !prevShowTable)`. React automatically passes the current state as the first parameter (`prevShowTable` - you can name it anything). This ensures you always get the latest state value, preventing bugs from rapid state updates. Use this pattern whenever your new state depends on the previous state.

The **fetchItems** (line16) and **useEffect** (line36) functions are unchanged.

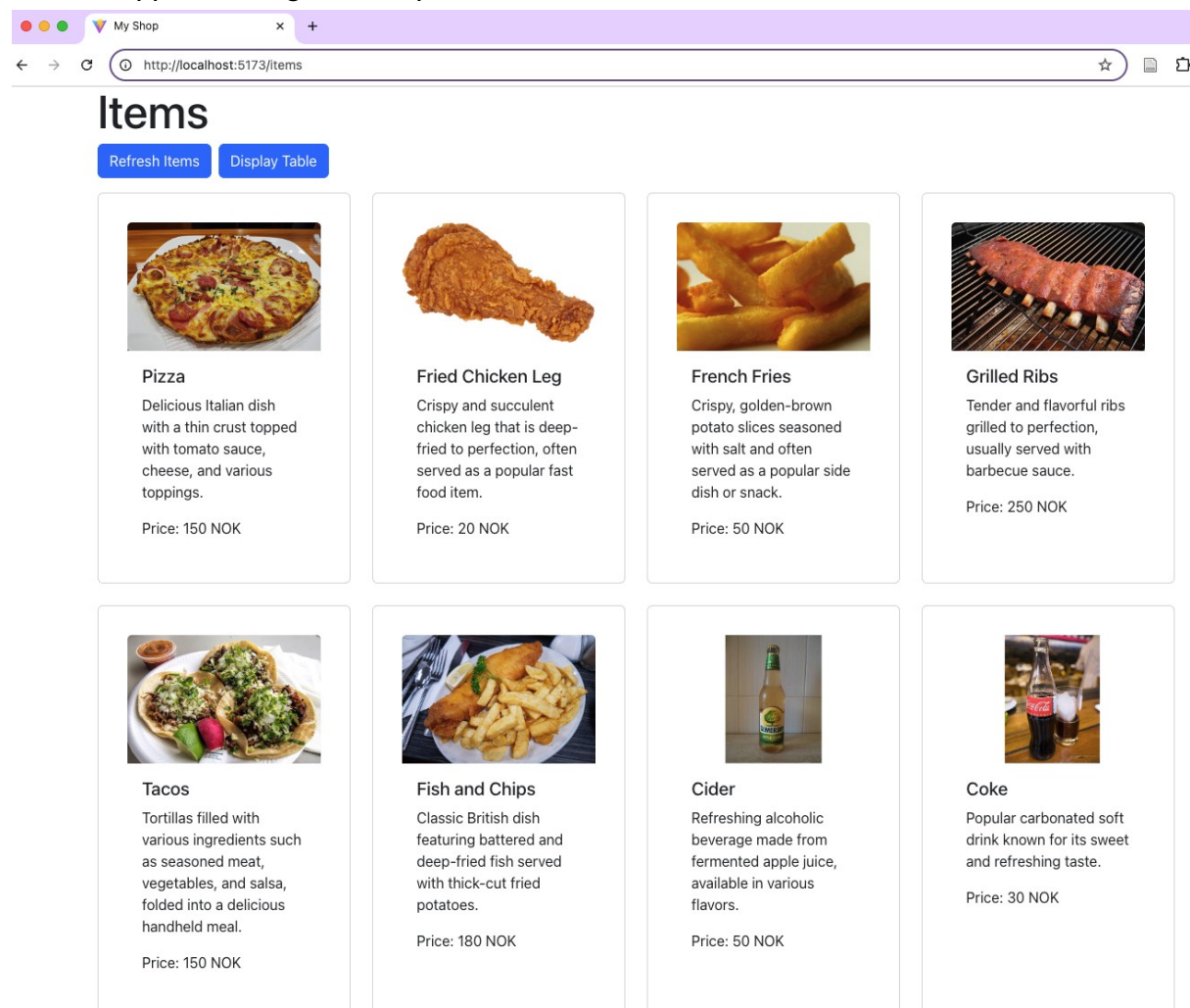
Line46 - Line48 adds an extra Button, clicking which will trigger the **toggleTableOrGrid** function. The button text is conditional rendered depending on the `showTable` variable.

Line50 - Line52 conditional renders the **ItemTable**, when **showTable** is true, or the **ItemGrid**, when **showTable** is false.

Now we run the project.

- Go to the terminal, `cd myshop`
- `npm install`
- `npm run dev`

The webapp is running as we expected:



MyShop Home Items Dropdown ▾

Items

Refresh Items Display Grid

ID	Name	Price	Descriptions	Images
1	Pizza	150 NOK	Delicious Italian dish with a thin crust topped with tomato sauce, cheese, and various toppings.	
2	Fried Chicken Leg	20 NOK	Crispy and succulent chicken leg that is deep-fried to perfection, often served as a popular fast food item.	
3	French Fries	50 NOK	Crispy, golden-brown potato slices seasoned with salt and often served as a popular side dish or snack.	
4	Grilled Ribs	250 NOK	Tender and flavorful ribs grilled to perfection, usually served with barbecue sauce.	
5	Tacos	150 NOK	Tortillas filled with various ingredients such as seasoned meat, vegetables, and salsa, folded into a delicious handheld meal.	
6	Fish and Chips	180 NOK	Classic British dish featuring battered and deep-fried fish served with thick-cut fried potatoes.	
7	Cider	50 NOK	Refreshing alcoholic beverage made from fermented apple juice, available in various flavors.	
8	Coke	30 NOK	Popular carbonated soft drink known for its sweet and refreshing taste.	

However, it may look not so nice for the table if we are using a small screen (click the button to enable mobile phone mode):

MyShop

Items

Refresh Items Display Grid

ID	Name	Price	Descriptions	Images
1	Pizza	150 NOK	Delicious Italian dish with a thin crust topped with tomato sauce, cheese, and various toppings.	
2	Fried Chicken Leg	20 NOK	Crispy and succulent chicken leg that is deep-fried to perfection.	

Mobile phone mode

Dimensions: Responsive 443 x 852 100% No throttling

Elements Console Sources

No Issues 9 hidden

Download the React DevTools for a better development experience: <https://react.dev/link/react-devtools>

Array(8) ItemListPage.jsx?t=1755161990

Array(8) ItemListPage.jsx?t=1755161990

Also, we would like to add a search mechanism if the item list is too long. We do these in the next section.

Fixing the lagging backend

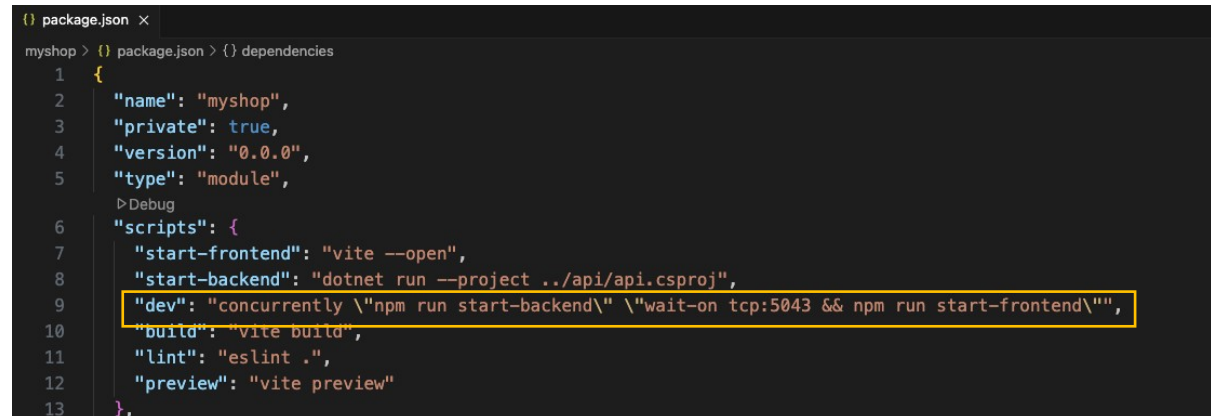
You may notice that sometimes your fetch items will fail because the frontend is running before your backend is ready. A simple fix is to let the frontend wait until the backend is ready. This is typically used in a development environment.

Go to myshop

```
cd myshop
```

```
npm install --save-dev wait-on
```

Then change the line9 in myshop/package.json to



```
() package.json x
myshop > {} package.json > {} dependencies
1  {
2    "name": "myshop",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "start-frontend": "vite --open",
8      "start-backend": "dotnet run --project ../api/api.csproj",
9      "dev": "concurrently \"npm run start-backend\" \"wait-on tcp:5043 && npm run start-frontend\"",
10     "build": "vite build",
11     "lint": "eslint .",
12     "preview": "vite preview"
13   },
```

This line of code make the frontend to start after waiting for the port to be open, regardless of HTTP responses. This is the most reliable for API backends that don't serve web pages at the root.

Then run

```
npm run dev
```

You will see that the frontend always starts after the backend is ready.

Toggling descriptions and images in ItemTable

We now change the ItemTable.jsx to (without stopping the debugging):

```
ItemTable.jsx x
myshop > src > items > ItemTable.jsx > ...
1  import React, { useState } from 'react';
2  import { Table, Button } from 'react-bootstrap';
3
4  const ItemTable = ({ items, apiUrl }) => {
5    const [showImages, setShowImages] = useState(true);
6    const [showDescriptions, setShowDescriptions] = useState(true);
7    const toggleImages = () => setShowImages(prevShowImages => !prevShowImages);
8    const toggleDescriptions = () => setShowDescriptions(prevShowDescriptions =>
    !prevShowDescriptions);
9
10   return (
11     <div>
12       <Button onClick={toggleDescriptions} className="btn btn-secondary mb-3 me-2">
13         {showDescriptions ? 'Hide Descriptions' : 'Show Descriptions'}
14       </Button>
15       <Button onClick={toggleImages} className="btn btn-secondary mb-3">
16         {showImages ? 'Hide Images' : 'Show Images'}
17       </Button>
18       <Table striped bordered hover>
19         <thead>
20           <tr>
21             <th>ID</th>
22             <th>Name</th>
23             <th>Price</th>
24             {showDescriptions && <th>Descriptions</th>}
25             {showImages && <th>Images</th>}
26           </tr>
27         </thead>
28         <tbody>
29           {items.map(item => (
30             <tr key={item.itemId}>
31               <td>{item.itemId}</td>
32               <td>{item.name}</td>
33               <td>{item.price} NOK</td>
34               {showDescriptions && <td>{item.description}</td>}
35               {showImages && <td><img src={` ${apiUrl}${item.imageUrl}`} alt={item.name}
36                 width="120" /></td>}
37             </tr>
38           ))}
39         </tbody>
40       </Table>
41     </div>
42   );
43
44   export default ItemTable;
```

Imports: import the hook **useState** and component **Button**

Line5-Line6: define two new states for toggling the images and descriptions.

Line7-Line8: define two functions for the toggling true/false value of **showImages** and **showDescriptions**.

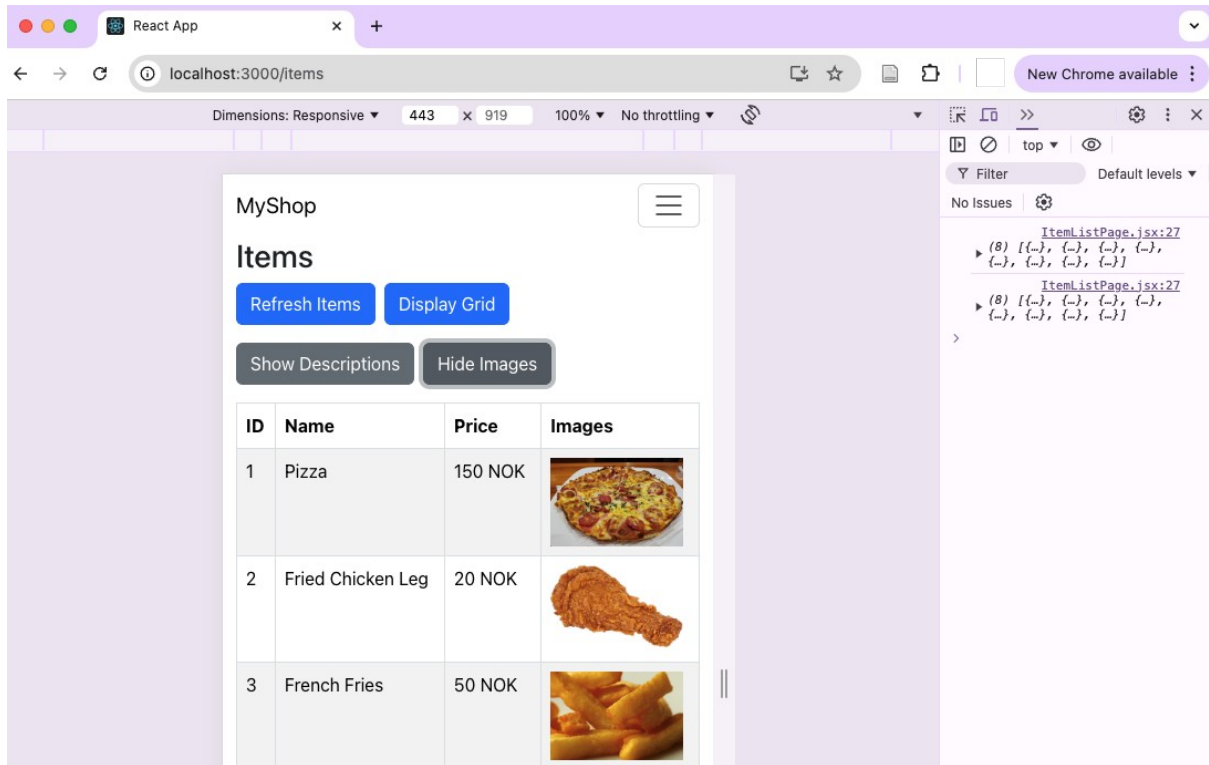
Line12-Line15: adding two buttons for toggling.

Line24-Line25: toggling the table headers.

Line34-Line35: toggling the table data.

Line11 and Line40: wrapping up the buttons and tables into a <div>, required by the return command.

We can see the effect immediately:



Filtering content

Now we add a search bar before the ItemTable or ItemGrid.

Change the ItemListPage.jsx to:

Line1-Line45

```
ItemListPage.jsx x
myshop > src > items > ItemListPage.jsx > ItemListPage
1  import React, { useState, useEffect } from 'react';
2  import { Button, Form } from 'react-bootstrap';
3  import ItemTable from './ItemTable';
4  import ItemGrid from './ItemGrid';
5
6  const API_URL = 'http://localhost:5043'
7
8  const ItemListPage= () => {
9    const [items, setItems] = useState([]);
10   const [loading, setLoading] = useState(false);
11   const [error, setError] = useState(null);
12   const [showTable, setShowTable] = useState(true);
13   const [searchQuery, setSearchQuery] = useState('');
14
15   const toggleTableOrGrid = () => setShowTable(prevShowTable => !prevShowTable);
16
17   const fetchItems = async () => { ...
35   };
36
37   useEffect(() => { ...
39   }, []);
40
41   const filteredItems = items.filter(item =>
42     item.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
43     item.description.toLowerCase().includes(searchQuery.toLowerCase())
44   );
45 }
```

Line2: imports **Form**, we need for inputting search query.

Line13: defines a new state, **searchQuery**.

Line17, Line37, no change.

Line41-Line44: defines a new const to store the result of the filtering operation. It calls the filter method on the items array, which creates a new array with all elements that pass a test implemented by the provided function.

item =>: Uses an arrow function as the test function for filtering. Each item in the items array is passed to this function.

The **toLowerCase()** makes the search case insensitive.

The **||** is logical OR operator, it means either the **item.name** or the **item.description** needs to contain the string in the **searchQuery**.

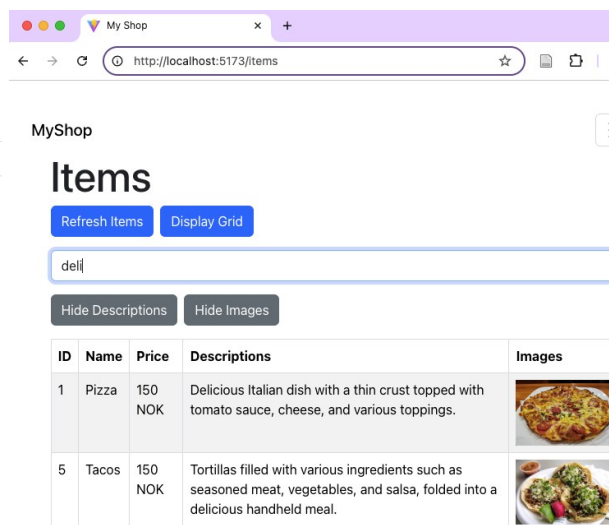
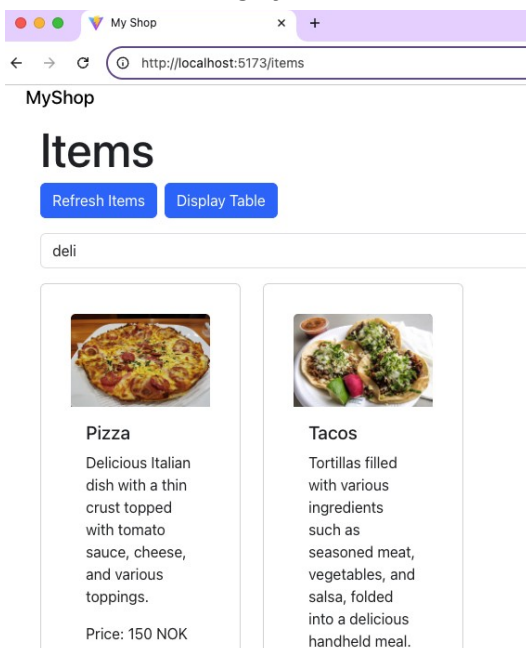
Line46-Line71:

```
46   return (
47     <div>
48       <h1>Items</h1>
49       <Button onClick={fetchItems} className="btn btn-primary mb-3 me-2" disabled={loading}>
50         {loading ? 'Loading...' : 'Refresh Items'}
51       </Button>
52       <Button onClick={toggleTableOrGrid} className="btn btn-primary mb-3 me-2">
53         {showTable ? 'Display Grid' : 'Display Table'}
54       </Button>
55       <Form.Group className="mb-3">
56         <Form.Control
57           type="text"
58           placeholder="Search by name or description"
59           value={searchQuery}
60           onChange={e => setSearchQuery(e.target.value)}
61         />
62       </Form.Group>
63       {error && <p style={{ color: 'red' }}>{error}</p>}
64       {showTable
65       ? <ItemTable items={filteredItems} apiUrl={API_URL} />
66       : <ItemGrid items={filteredItems} apiUrl={API_URL} />}
67     </div>
68   );
69 };
70
71 export default ItemListPage;
```

Line55-Line62, defines an input Form for inputting the **searchQuery**. The default text is rendered by Line58, and Line60 sets the value of the state **searchQuery** to the input text from the form once the input **Form** changes.

Line65-Line66: pass the **filteredItems** to the **ItemTable** or **ItemGrid**, not the complete array of items.

We see that the search bar is applied to both **ItemTable** and **ItemGrid**, because it is written on the **ItemListPage.jsx**, which contains the general logic for the components on the page.



Carousel in HomePage

In the end of this demo, we create the Carousel on the HomePage.jsx.

```
HomePage.jsx x
myshop > src > home > HomePage.jsx > ...
1  import React from 'react';
2  import Carousel from 'react-bootstrap/Carousel';
3  import Image from 'react-bootstrap/Image';
4
5  const API_URL = 'http://localhost:5043'
6
7  const HomePage = () => {
8    return (
9      <div className="text-center">
10        <h1 className="display-4">Welcome to My Shop</h1>
11        <Carousel>
12          <Carousel.Item>
13            <Image src={` ${API_URL}/images/pizza.jpg`} className="d-block w-100"
14              alt="Pizza" />
15          </Carousel.Item>
16          <Carousel.Item>
17            <Image src={` ${API_URL}/images/fishandchips.jpg`} className="d-block w-100"
18              alt="Fish and Chips" />
19          </Carousel.Item>
20          <Carousel.Item>
21            <Image src={` ${API_URL}/images/tacos.jpg`} className="d-block w-100"
22              alt="Tacos" />
23          </Carousel.Item>
24        </Carousel>
25      </div>
26    );
27  };
28
29 export default HomePage;
```

Very convenient to use the Carousel and Image components from react-bootstrap. You may notice that we need the **API_URL** again, which is quite cumbersome. We will fix that later.

Congratulations! You have finished the demo of conditional rendering! 😊